B.Tech. Sem-V

Fundamentals of Operating System Design (CEUC301)

UNIT 8

I/O System Design and Disk Scheduling

Conceptual, Analytical, Scenario-Based and HOTS Question Bank

Syllabus Coverage

Interrupt handling; Direct Memory Access (DMA) design; buffering and buffer cache; I/O scheduling; disk scheduling algorithms: FCFS, SSTF, SCAN, LOOK, C-SCAN, and C-LOOK.

A. Interrupt Handling and Interrupt Architecture

1. Explain the role of interrupts in an operating system and distinguish interrupt-driven I/O from programmed I/O.
2. Differentiate hardware interrupts, software interrupts, exceptions, and traps, with an example of each.
3. Describe the sequence of events that occurs from the arrival of an interrupt to the resumption of the interrupted process.
4. Why does a processor save execution context during interrupt handling? Which categories of state may need to be preserved?
5. Explain the role of an interrupt service routine (ISR). Why should an ISR generally execute quickly?
6. Differentiate interrupt latency from interrupt service time and explain why both matter in a real-time system.
7. What is interrupt priority? Explain how priority levels can influence responsiveness when several devices request service.
8. Compare vectored and non-vectored interrupts. What design trade-offs do they introduce?
9. Explain nested interrupts and identify circumstances in which allowing nesting may be beneficial or risky.
10. How can interrupt storms or excessive interrupt rates degrade overall system performance? Suggest operating-system-level mitigations.

B. Direct Memory Access (DMA) Design

11. Explain the purpose of DMA and how it reduces CPU involvement in data transfers.
12. Describe the typical steps required to initiate, execute, and complete a DMA transfer.
13. Compare programmed I/O, interrupt-driven I/O, and DMA in terms of CPU overhead, transfer size, latency, and complexity.
14. What responsibilities remain with the CPU and device driver when DMA is used?
15. Explain why DMA transfers require coordination between a device controller, system memory, and the operating system.
16. Distinguish burst mode, cycle stealing, and transparent DMA at a conceptual level.
17. How can cache coherence problems arise when a device uses DMA to read or write memory? What approaches can address them?
18. Explain the role of IOMMUs and DMA mapping in protecting memory and supporting devices with address constraints.
19. What can go wrong if a DMA buffer is released or reused before the device has completed the transfer?
20. An application transfers large blocks from a high-speed storage device. Evaluate whether DMA is preferable to interrupt-driven I/O and justify your choice.

C. Buffering, Buffer Cache, and Asynchronous I/O

21. Define buffering and explain why it is useful when a producer and consumer operate at different speeds.
22. Distinguish single buffering, double buffering, and circular buffering, and identify a suitable use case for each.
23. Explain how buffering can overlap computation with I/O and improve resource utilization.
24. Differentiate buffering, caching, and spooling. Why are these mechanisms not interchangeable?
25. What is a buffer cache? Explain how it can reduce physical disk I/O.
26. Explain cache hits and cache misses in the context of a buffer cache. Which workload characteristics tend to improve the hit rate?
27. Compare write-through and write-back caching, including their performance and data-integrity implications.
28. Why can dirty buffers create consistency or durability risks during a crash or power failure?
29. Explain the purpose of cache replacement policies and why recency, frequency, and workload patterns matter.
30. How can asynchronous I/O improve application responsiveness? What synchronization issues can arise when an I/O request completes later?
31. Explain the difference between blocking and non-blocking I/O, and between synchronous and asynchronous I/O.
32. Discuss how buffer size affects throughput, memory consumption, latency, and the number of I/O operations.

D. I/O Scheduling and Device Queue Design

33. Why do operating systems schedule I/O requests instead of always servicing them strictly in arrival order?
34. Explain the objectives of I/O scheduling, including throughput, response time, fairness, predictability, and device utilization.
35. How can request merging and request reordering reduce storage-device overhead?
36. Explain the trade-off between maximizing throughput and providing fair service to all processes.
37. How can an I/O scheduler cause starvation? Identify design techniques that can reduce the risk.
38. Why do scheduling policies for rotating disks differ from policies for SSDs and NVMe devices?
39. Explain how queue depth can affect storage throughput and latency. Why can an excessively deep queue be harmful?
40. Discuss how workload type—sequential, random, read-heavy, or write-heavy—can influence I/O scheduling decisions.

E. Disk Scheduling Algorithms

41. Explain the operating principle of FCFS disk scheduling and discuss its strengths and weaknesses.
42. How does SSTF select the next request? Explain why it can reduce average seek distance yet still produce starvation.
43. Describe SCAN scheduling and explain the elevator analogy commonly used to understand it.
44. Compare SCAN and LOOK. Why can LOOK avoid unnecessary movement toward the physical end of the disk?
45. Explain C-SCAN and how its service pattern differs from SCAN.
46. Compare C-SCAN and C-LOOK, focusing on the direction of service and the distance traversed between request groups.
47. Compare FCFS, SSTF, SCAN, LOOK, C-SCAN, and C-LOOK in terms of seek efficiency, fairness, and response-time predictability.
48. Why can SSTF produce a different fairness profile from SCAN or C-SCAN under a continuous stream of requests?
49. Under what request-arrival patterns might FCFS be a reasonable choice despite potentially higher head movement?
50. Why should a disk scheduling algorithm be evaluated using more than average seek distance alone?
51. How do the assumptions behind classical disk scheduling algorithms change when the storage device is an SSD?
52. Explain why C-SCAN and C-LOOK are often discussed as ways to provide more uniform waiting times than direction-reversing policies.

F. Integrative and Higher-Order Thinking (HOTS)

53. A server handles network packets, storage requests, and sensor input. Propose how interrupts, DMA, buffering, and I/O scheduling could work together in its I/O path.

54. A device generates interrupts at a very high rate. Analyze the performance risks and recommend a combination of interrupt moderation, DMA, and buffering strategies.
55. A storage service achieves high throughput but exhibits poor tail latency. Identify possible causes involving queue depth, caching, buffering, and I/O scheduling.
56. A system loses recently written data after a sudden power failure. Analyze how write-back caching and dirty-buffer handling may contribute, and propose safeguards.
57. A workload contains many nearby disk requests but occasional requests from distant cylinders. Compare SSTF and LOOK for this workload, considering both efficiency and fairness.
58. An embedded system must respond predictably to urgent device events while also transferring large data blocks. Recommend an interrupt and DMA design, and justify the trade-offs.
59. Critically evaluate whether one I/O scheduling policy should be used for every storage device and workload.
60. Design a conceptual test plan for comparing FCFS, SSTF, SCAN, LOOK, C-SCAN, and C-LOOK without relying on a single performance metric.